

Exercises for Weeks 5–6

Programming DT2012

Fall 2025

1 Introduction

This assignment focuses on **using abstract data types** with colors and images as examples, and **designing our own abstract data types**. In this case, we will use geographic coordinates. Each part has two activities that you need to complete. These exercises cover the topics from lectures 5 and 6. All instructions for each activity are in sections 2 and 3.

After completing the assignment, you will submit your files through Blackboard. The submission package **must** be a compressed directory (i.e., a zip file) with **only** the Java files for each activity within it. The compressed directory **must** have the same name as your student acronym (i.e., magols25), and the Java files **must** be called A1.java, A2.java (for activities 1 and 2, respectively), Coordinate.java for activity 3 and Trajectory.java for activity 4.

2 Using ready-made abstract data types for image processing

Objectives

By the end of this part of the assignment:

- You will be able to use previously designed abstract data types that define colors and pictures.
- You will be able to use these abstract data types to create and modify digital pictures.

These exercises will also allow you to use your skills for creating and manipulating arrays, which are the basis of digital pictures — they are basically a 2D array of pixels with different colors.

Activity 1: altering the colors in an image

Image processing is an interesting area for computer science. Using digital tools, a picture can be enhanced and usable, e.g., in computer vision. Some of these

tools include turning an image into black and white, turning it into a negative, rotating it and mirroring it.

Using the *Color* class from Java, and the *Luminance.java* class that we provide you with, you can use and create procedures that will help you process an image which we also provide.

Write a program that takes the file name for an image as an argument and produces its gray scale version, a full negative version (you can find a full definition in [this Wikipedia article](#)), a rotated image (90° clockwise), and a flopped version.

Requirements and Constraints

- The program will take the file name (e.g., *baboon.jpg*) as an input.
- The program will create, show, and save the following pictures:
 - A gray scale version (according to what you saw in the lectures) saved as *pic1.jpg*.
 - A full negative version (complement of colors) saved as *pic2.jpg*.
 - A version of the image rotated 90° clockwise saved as *pic3.jpg*
 - A flopped version (mirrored in the vertical axis) saved as *pic4.jpg*

Example run

These instructions are valid for UNIX-based systems. For Windows, *ls* is replaced by *dir*.

```
$ ls
A1.java A1.class baboon.jpg
$ java A1 baboon.jpg
$ ls
A1.java A1.class baboon.jpg pic1.jpg pic2.jpg
pic3.jpg pic4.jpg
```

The images shall comply with what is expected by definition. The automatic test will compare the images against our models shown in Fig. 1.

Activity 2: image filters

Image filters help create effects on images that enhance details or direct the viewer's focus to a certain part of the image. Some common filters blur or sharpen an image or darken the corners to highlight the center (i.e., [vignetting](#))

A blurred version of a digital image is obtained when the color in each pixel is a color with components obtained as the average of the components of the 9 colors around (and including) the pixel.

A sharpened version of the image where the color in each pixel is a color with components obtained as the [weighted average](#) of the components of the 9

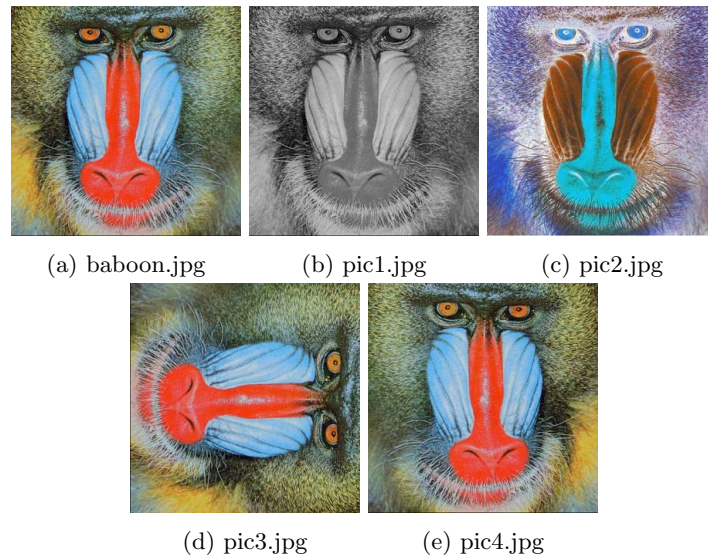


Figure 1: The original picture and our models

colors around (and including) the pixel. The weights are: 16 for the pixel itself and -1 for the 8 neighbors.

A vignetted image is obtained by mixing the pixels in the picture with black in a proportion where the pixel at the center is the original color and the pixels at the border are totally black. The pixels in between will get mixed with black by giving that color an *alpha* that increases proportionally to the distance to the center of the picture. For this, you have to calculate the maximum distance from the center, the ratio between the distance of a pixel and the maximum distance, and apply that ratio as the alpha for black in the mix (alpha can go from 0 to 1).

Write a program that takes the file name for an image as an argument and produces its blurred, sharpened, and vignetted versions according to the definitions and characteristics we provided.

Requirements and Constraints

- The program will take the file name (e.g., *baboon.jpg*) as an input.
- The program will create, show, and save the following pictures:
 - A blurred version saved as *pic5.jpg*.
 - A sharpened version saved as *pic6.jpg*.
 - A vignetted version saved as *pic7.jpg*
- To obtain the averages and weighed averages required for blurring and sharpening, you must use your *NumericalArrays.java* class from Weeks

3–4.

- The only pre-defined classes you can use are your *NumericalArrays* class and the *StdDraw*, *Luminance*, and *Picture* classes that we provide you (those specific versions). The only import you can use is *import java.awt.Color*.

Example run

These instructions are valid for UNIX-based systems. For Windows, *ls* is replaced by *dir*.

```
$ ls
A1.java A1.class A2.java A2.class baboon.jpg pic1.jpg
pic2.jpg pic3.jpg pic4.jpg
$ java A2 baboon.jpg
$ ls
A1.java A1.class A2.java A2.class baboon.jpg pic1.jpg
pic2.jpg pic3.jpg pic4.jpg pic5.jpg pic6.jpg pic7.jpg
```

The images shall comply with what is expected by definition. The automatic test will compare the images against our models shown in Fig. 2.

3 Creating and using abstract data types from scratch

Activity 3: coordinate systems, distance, and movement

The definition of a geographic coordinate distance to Wikipedia is:

A geographic coordinate system (GCS) is a spherical or geodetic coordinate system for measuring and communicating positions directly on Earth as latitude and longitude. It is the simplest, oldest and most widely used type of the various spatial reference systems that are in use, and forms the basis for most others. Although latitude and longitude form a coordinate tuple like a cartesian coordinate system, the geographic coordinate system is not cartesian because the measurements are angles and are not on a planar surface.

You have most likely used coordinates when you calculate the distance between two places using a Maps application, or even an app for fitness and running. In doing so, you have probably noticed that coordinates are sometimes written as decimal numbers with (+) and (-) signs, or as sexagesimal numbers (with degrees, minutes, and seconds) with markers for North, South, East and West. The [Wikipedia page for coordinate conversion](#) delves deeper in the different ways coordinates can be expressed.

In a similar fashion to what we did with the *Complex* class during the lectures (i.e., where we developed an abstract data type to deal with complex numbers

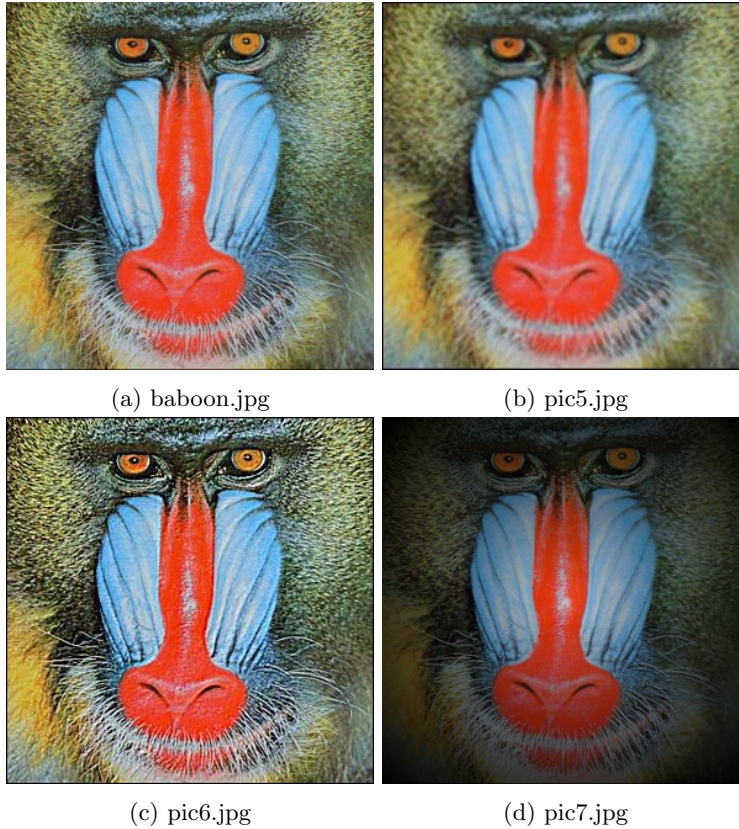


Figure 2: The original picture and our models

and operations), you will complete the *Coordinate* class that we provide you with in this package. So far, the class has the attributes for a coordinate (i.e., latitude and longitude), and a constructor that takes two double values, checks if they are within the constraints (i.e. latitude range is $[-90^\circ, 90^\circ]$ and longitude range is $[-180^\circ, 180^\circ]$), and prints them as a string in decimal form.

Complete the *Coordinate* class so that it can also print the coordinates in sexagesimal degrees and the distance between two points using the Haversine formula (see example). The program must use the template provided and must not import any other libraries or classes.

Requirements and Constraints

- The program will have a *main* method that shows the capabilities of the program.
- The main method is already given and it is expected to deliver the appro-

ropriate output (which means the methods are appropriately implemented).

- The only external library you can use is *Math*.
- The abstract data type must be able to be used from another program (e.g., *TestCoordinate.java*), where we can create several coordinates, print them in different formats, and calculate the distance between them. This is how we are going to test the program functionality.

Example run

```
$ java Coordinate
Halmstad is at 56.6744° N, 12.8578° E or 56° 40' 27" N, 12° 51' 28" E
Helsinborg is at 56.0465° N, 12.6945° E or 56° 2' 47" N, 12° 41' 40" E
Distance: 70.54023629909423 km
```

Activity 4: paths and trajectories

In this activity, you will combine the skills you have developed through the course: working with different data types, working with arrays, reading information from files, and drawing. You have gotten a file with the coordinates for a set of places, the first one is the latitude and longitude for Halmstad, and the rest are the latitudes and longitudes for the capital cities of the 27 countries in the European Union.

Create a program that reads a file with the coordinates of n cities (with n being the first number in the file, and the next two numbers representing the "anchor" city). Then, the program calculates a path that goes through every city always choosing the closest next hop without repeating cities (i.e., Halmstad is our anchor city and it will pick Copenhagen, then Copenhagen chooses the closest city to it except for Halmstad). Then, the path is drawn using the *StdDraw* class with points and lines.

Requirements and Constraints

- The program will receive a file with coordinates as an input. The first value in the program (an integer) is the number of cities including the anchor, and the first pair of double values will be used to create the coordinates.
- Using the *Coordinate* class, the program constructs all the sets of coordinates and stores them, e.g., in an array.
- The first position of in the trajectory is the anchor city (first set of coordinates). This is, e.g., step 0.
- The first hop then determines its next hop: the city closest to it that has not been visited yet. When found, it stores it in the trajectory array (as the step 1).

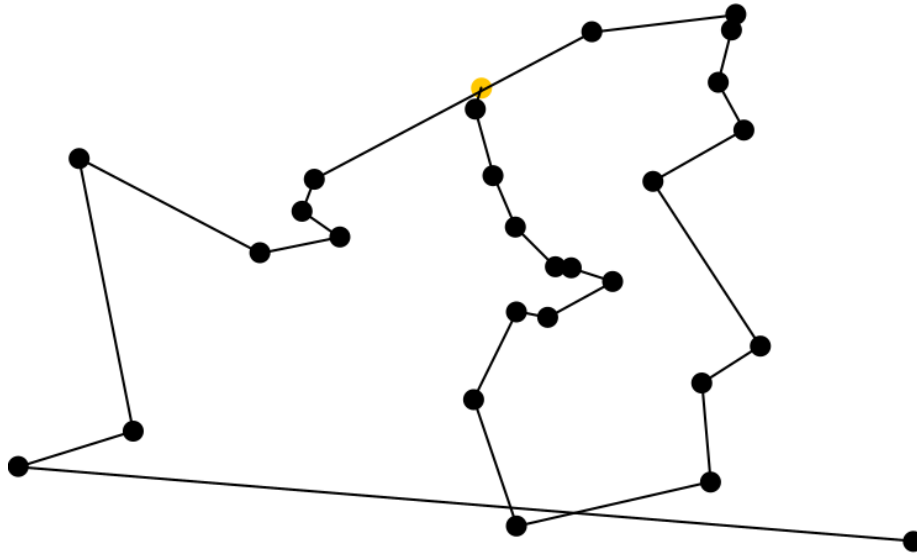


Figure 3: trajectory.png

- The process is repeated until every city has been visited.
- The trajectory array is then drawn using *StdDraw* where the coordinates are represented as dots, and the trajectory as lines connecting these dots. The point for the first city is painted with a different color. All dots have the same radius (10) and all lines have the same width (2). This picture is saved as *trajectory.jpg*.
- You are only allowed to use *StdDraw*, *Picture*, and *Math*. No other pre-defined functions are allowed and the use of them will render your submission as failed.

Example run

```
$ java Trajectory < eucapitals.txt
$ ls
A1.java A1.class A2.java A2.class Coordinate.java
Coordinate.class Trajectory.java Trajectory.class
eucapitals.txt baboon.jpg pic1.jpg
pic2.jpg pic3.jpg pic4.jpg pic5.jpg pic6.jpg pic7.jpg
trajectory.jpg
```

Fig. 3 shows a possible result of the execution of the Trajectory program. Notice how every point is only visited once and that the next hop is always chosen as the closest one to the current hop (not the closest one to the first point).

4 Grading criteria

Each activity is evaluated as a unit. There are 4 points at play, and you must get at least 3 to pass this set of exercises (75%). To pass each unit, the program must compile and deliver the expected output depending on the arguments that are provided. The code must comply with **all** the requirements and constraints described in the activity.

To be awarded the points, you must hand in original code that was designed and typed by yourself. Code that does not pass the similarity test we subject it to will be submitted to the disciplinary board. Even if you work in teams or collaborate, each student must submit original code. Code that was obtained from Generative AI assistants is not original code.

Points are awarded after the mandatory oral examination. In this examination, you will answer specific questions about your code, explain how changing some commands affects performance, and general questions about Java and programming. You can bring your own pseudocode or activity diagrams to the oral examination as aids.

Tasks from old exams that can be solved after week 6.

1. This exercise deals with the data type `Balloon` with the following implementation:

```
public class Balloon{

    private double centerX;
    private double centerY;
    private double rad;

    public Balloon(double x, double y, double r){
        centerX = x;
        centerY = y;
        rad = r;
    }

    public void draw(){
        StdDraw.filledCircle(centerX, centerY, rad);
    }
}
```

- (a) Add a method to blow the balloon up by increasing its radius by 10. The method shall have this signature:

```
public void blowUp()
```

The method changes the instance variable for the radius by increasing its value by 10:

```
public void blowUp(){
    rad = rad + 10;
}
```

- (b) Add a method to make the balloon fly by increasing the vertical component for the balloon's center. The method shall have this signature

```
public void fly(double step)
```

The method changes the instance variable for the y-coordinate of the midpoint by increasing adding to it the value of the `step` argument:

```
public void fly(double step){
    centerY = centerY + step;
}
```

2. This assignment deals with the data type `BankAccount` with the following implementation:

```

public class BankAccount{
    private String personNumber;
    private int saldo;

    public BankAccount(String pn){
        personNumber = pn;
        saldo = 0;
    }

    public String toString(){
        return "Account belonging to " + personNumber + " with saldo: " + saldo;
    }

    public static void main(String[] args){
        BankAccount me = new BankAccount("111111-1111");
        me.deposit(1000);
        System.out.println(me);
    }
}

```

- (a) Add a method that adds money to a bank account to that the balance increases:

```
public void deposit(int value)
```

Solution suggested:

```

public void deposit(int value){
    saldo = saldo + value;
}

```

The instance variable is updated by adding `value`.

- (b) Create a method to decrease the balance in an account when a withdrawal occurs:

```
public void retrieve(int value)
```

Solution suggested:

```

public void deposit(int value){
    if(saldo >= value){saldo = saldo - value;}
}

```

The instance variable is updated by subtracting the amount specified by `value` if there is enough money in the account.